TP3 – CI3 : Réseaux convolutifs

Maroua Zoubir
Télécom SudParis – CSC 8607, Introduction au deep learning

18 octobre 2026

1 Exercice 1 – Préparation d'ImageNet et augmentation de données

Code complété

1.1 Que se passe-t-il sans `Normalize` ? Pourquoi calculer les statistiques sur le train uniquement ?

La normalisation transforme chaque canal RGB selon

$$x' = \frac{x - \mu}{\sigma},$$

où μ et σ sont la moyenne et l'écart-type du canal.

Sans normalisation. Après `ToTensor()`, les pixels sont dans $[0, 1]$, tous positifs et de moyenne voisine de 0,45 (non centrés). Cela a plusieurs conséquences.

— **Signe du gradient identique pour tous les poids d'un neurone.** Pour un neurone de la première couche, $\dfrac{\partial \mathcal{L}}{\partial w_i} = \delta\, x_i$ avec $x_i \geq 0$: tous les $\partial \mathcal{L}/\partial w_i$ ont le signe de δ. Les poids d'un même neurone ne peuvent que monter ensemble ou descendre ensemble à chaque pas, d'où une trajectoire en zigzag dans l'espace des poids.

— **Mauvais conditionnement.** Des entrées non centrées et d'échelles différentes selon les canaux rendent la hessienne mal conditionnée : la surface de perte est très allongée, la descente de gradient converge lentement et impose un learning rate plus petit pour rester stable.

— **Incohérence avec l'initialisation.** Les initialisations (Kaiming, Xavier) supposent des entrées de moyenne nulle et de variance proche de 1. Sinon, la variance des pré-activations dérive de couche en couche et l'échelle des activations et des gradients est altérée.

Cela ne signifie pas que l'entraînement échoue nécessairement, mais il est plus lent et plus sensible au learning rate.

Statistiques calculées sur le train uniquement. Le jeu de validation (et de test) simule des données *jamais vues*. Utiliser ses propres statistiques ferait fuiter de l'information du jeu d'évaluation vers le pipeline d'entraînement (*data leakage*) et biaiserait l'estimation de la généralisation. En production, on ne connaît pas les statistiques du futur jeu de test : on applique donc au val/test *les mêmes constantes* que celles du train. Dans ce TP, on utilise les statistiques standard d'ImageNet.

1.2 Pourquoi `RandomResizedCrop` et `RandomHorizontalFlip` au train, mais `Resize` + `CenterCrop` en validation ?

— **Train.** Le but est de *régulariser*. À chaque époque, le réseau voit une version différente de chaque image (région, échelle, orientation). Cela augmente la diversité des données et impose des invariances (translation, échelle, symétrie gauche-droite), ce qui limite le sur-apprentissage d'un réseau à environ 61 M de paramètres.

— **Validation.** Le but est de *mesurer*, ce qui exige un protocole déterministe, reproductible et comparable d'une époque et d'un modèle à l'autre. Des transformations aléatoires ajouteraient du bruit à la métrique. `Resize(256)` puis `CenterCrop(227)` donne une vue standard centrée ; le rapport $227/256 \approx 0{,}89$ conserve une échelle apparente des objets cohérente avec celle de l'entraînement.

1.3 Exemple où `RandomHorizontalFlip` est néfaste

La classification de **panneaux de signalisation directionnels** : un panneau « tourner à gauche » retourné horizontalement devient un panneau « tourner à droite ». Si l'étiquette n'est pas modifiée, le modèle reçoit une image dont le contenu ne correspond plus au label, ce qui introduit du bruit d'étiquetage et dégrade les performances. Même problème pour la reconnaissance de caractères (« b » et « d », « p » et « q ») ou l'imagerie médicale où la latéralité compte.

2 Exercice 2 – Étude théorique et implémentation (MLP vs AlexNet)

2.0.1 Nombre de paramètres du MLP

Le MLP comporte trois couches linéaires $154\,587 \to 256 \to 256 \to 1000$, avec $3 \times 227 \times 227 = 154\,587$ valeurs en entrée. Pour une couche linéaire : avec biais $N_{in}N_{out}+N_{out}$, sans biais $N_{in}N_{out}$.

Couche	Avec biais	Sans biais
Linear 1 : $154\,587 \to 256$	$154\,587 \times 256 + 256 = 39\,574\,528$	$39\,574\,272$
Linear 2 : $256 \to 256$	$256 \times 256 + 256 = 65\,792$	$65\,536$
Linear 3 : $256 \to 1000$	$256 \times 1000 + 1000 = 257\,000$	$256\,000$
Total	**39 897 320**	**39 895 808**

La première couche concentre

$$\frac{39\,574\,528}{39\,897\,320} \times 100 \approx 99{,}19\,\%$$

des paramètres, soit plus de 99 %.

2.0.2 Défaut structurel majeur du MLP

Le `flatten` **détruit la structure spatiale 2D** de l'image : les pixels sont transformés en un vecteur sans que leurs relations de voisinage soient préservées. Chaque neurone est connecté à tous les pixels avec des poids indépendants : il n'y a ni localité des connexions, ni partage de poids. Le MLP n'est donc pas invariant (ni équivariant) par translation : un objet décalé de quelques pixels active des poids tout à fait différents, et le même motif doit être réappris à chaque position. Il généralise mal et sur-apprend facilement.

2.1 Étape 2 : AlexNet

Dimensions : $H_{out} = \left\lfloor \frac{H_{in} + 2P - K}{S} \right\rfloor + 1$. Paramètres d'une convolution : $(C_{in} K_H K_W + 1)\, C_{out}$. Mémoire : $C \times H \times W \times 4$ octets, divisés par 1024 pour obtenir des KB.

Couche	Entrée (C, H×W)	Hyperparamètres	Sortie (C, H×W)	# Param. (avec biais)	Mém. sortie (KB)
Conv1 + ReLU	3, 227×227	64 filtres, 11×11, S=4, P=2	64, 56×56	23 296	784
MaxPool1	64, 56×56	3×3, S=2, P=0	64, 27×27	0	182,25
Conv2 + ReLU	64, 27×27	192 filtres, 5×5, S=1, P=2	192, 27×27	307 392	546,75
MaxPool2	192, 27×27	3×3, S=2, P=0	192, 13×13	0	126,75
Conv3 + ReLU	192, 13×13	384 filtres, 3×3, S=1, P=1	384, 13×13	663 936	253,5
Conv4 + ReLU	384, 13×13	256 filtres, 3×3, S=1, P=1	256, 13×13	884 992	169
Conv5 + ReLU	256, 13×13	256 filtres, 3×3, S=1, P=1	256, 13×13	590 080	169
MaxPool3	256, 13×13	3×3, S=2, P=0	256, 6×6	0	36
Flatten + Dropout	256, 6×6	$p = 0{,}5$	9 216 (1D)	0	36
Linear1 + ReLU + Dropout	9 216	4096 neurones ($p = 0{,}5$)	4 096	37 752 832	16
Linear2 + ReLU	4 096	4096 neurones	4 096	16 781 312	16
Linear3 (sortie)	4 096	1000 classes	1 000	4 097 000	3,91
Total				**61 100 840**	**2 339,16**

Détail des calculs (exemples) :

— Conv1 : $(227 + 2 \cdot 2 - 11)/4 + 1 = 56$; paramètres $(3 \times 11 \times 11 + 1) \times 64 = 23\,296$; mémoire $\frac{64 \times 56 \times 56 \times 4}{1024} = 784$ KB.

— MaxPool1 : $\lfloor (56 - 3)/2 \rfloor + 1 = 27$; MaxPool2 : $\lfloor (27 - 3)/2 \rfloor + 1 = 13$; MaxPool3 : $\lfloor (13 - 3)/2 \rfloor + 1 = 6$, d'où $256 \times 6 \times 6 = 9\,216$ valeurs.

— Conv2 : $(27 + 4 - 5)/1 + 1 = 27$; Conv3 à Conv5 conservent 13×13 ($K = 3$, $P = 1$).

— Linear1 : $9\,216 \times 4\,096 + 4\,096 = 37\,752\,832$; Linear2 : $4\,096^2 + 4\,096 = 16\,781\,312$; Linear3 : $4\,096 \times 1\,000 + 1\,000 = 4\,097\,000$.

— Total des convolutions : 2 469 696 ; total des couches denses : 58 631 144.

Remarque : le tableau compte la sortie du bloc Flatten + Dropout comme une activation supplémentaire de 36 KB. En pratique, l'aplatissement est une simple vue mémoire sans copie.

2.1.1 Où se concentrent les paramètres et la mémoire des activations ?

— **Paramètres** : dans les **couches entièrement connectées**. Linear1 possède à elle seule 37 752 832 paramètres, soit $\approx 61{,}8\,\%$ du total ; les trois couches linéaires représentent $\approx 96\,\%$ des paramètres. Les cinq convolutions n'en représentent que $\approx 4\,\%$.

— **Activations** : dans les **premières couches convolutives**, qui produisent de grandes cartes de caractéristiques : Conv1 + MaxPool1 + Conv2 = 784 + 182,25 + 546,75 = 1 513 KB, soit $\approx 65\,\%$ du total (Conv1 seule $\approx 34\,\%$). Les couches denses ne produisent que quelques dizaines de KB. Ces activations doivent être conservées (ou recalculées) pendant le forward pour permettre la rétropropagation.

2.1.2 Mémoire d'activations : AlexNet vs MLP

Pour le MLP, les sorties des trois couches représentent $256 + 256 + 1\,000 = 1\,512$ valeurs, soit

$$\frac{1\,512 \times 4}{1024} \approx 5{,}9 \text{ KB}.$$

AlexNet en stocke environ 2 339 KB, soit ≈ 400 fois plus, alors que les nombres de paramètres sont du même ordre de grandeur (61,1 M contre 39,9 M).

Convention de comptage : on a ici compté uniquement les sorties des couches pour les deux réseaux. Si l'on ajoute l'image d'entrée des deux côtés ($154\,587 \times 4/1024 \approx 603{,}86$ KB), on obtient $\approx 609{,}8$ KB pour le MLP et $\approx 2\,943$ KB pour AlexNet, soit un rapport d'environ 4,8. Dans les deux conventions, AlexNet consomme nettement plus de mémoire d'activations.

Explication. Le nombre de paramètres et la taille des activations sont deux grandeurs indépendantes.

— Dans le MLP, chaque poids n'est utilisé qu'*une fois* par image ; les couches cachées sont étroites (256), donc les activations sont minuscules, mais presque tous les poids sont dans l'énorme matrice d'entrée.

— Dans une convolution, un même filtre est *partagé* et appliqué à toutes les positions spatiales : peu de paramètres, mais une sortie $C \times H \times W$ volumineuse. La mémoire d'activations dépend de la résolution et du nombre de canaux, pas du nombre de poids.

— Pratiquement, avec un batch de 256 images, AlexNet demande $\approx 2\,339 \times 256 \approx 585$ MB d'activations, à ajouter aux poids (61,1 M $\times$ 4 $\approx$ 233 MB), aux gradients et aux moments d'Adam.

Conclusion : un nombre de paramètres similaire n'implique pas une consommation mémoire similaire.

2.2 Vérification avec `count_parameters`



FIGURE 1 – Sortie de `count_parameters` sur le cluster.

Les valeurs obtenues (MLP : 39 897 320 ; AlexNet : 61 100 840) sont identiques aux calculs théoriques.

3 Exercice 3 – Boucle d'entraînement, Early Stopping et comparaison rapide

Remarque : dans l'énoncé, la variable est écrite `run_ tag` (avec une espace) ; il faut écrire `run_tag`.

3.1 Rôle de `model.eval()` et `model.train()` vis-à-vis de `nn.Dropout`

— `model.train()` : chaque `nn.Dropout(p=0.5)` met à zéro chaque activation avec probabilité 0,5 et multiplie les autres par $1/(1-p) = 2$ (*inverted dropout*), pour conserver la même espérance de sortie. Cela régularise le réseau en empêchant les co-adaptations de neurones.

— `model.eval()` : le Dropout devient l'identité ; toutes les unités sont actives et la sortie est déterministe.

— **Si l'on oublie `model.eval()` en évaluation** : la moitié des neurones des couches denses serait encore éteinte au hasard. Les prédictions seraient bruitées et non reproductibles (deux évaluations successives donneraient des résultats différents), la loss de validation serait artificiellement plus haute et l'accuracy plus basse. On obtiendrait une mauvaise estimation de la performance et un Early Stopping potentiellement déclenché à tort. `torch.no_grad()` ne change rien à ce comportement : il ne gère que le calcul des gradients.

3.2 Valeurs théoriques à l'initialisation

Avec 1000 classes équilibrées et un choix uniformément aléatoire :

$$\text{accuracy} = \frac{1}{1000} = 0{,}1\,\%, \qquad \mathcal{L}_{CE} = -\ln \frac{1}{1000} = \ln(1000) \approx 6{,}908.$$

Run	Train loss	Val loss	Val acc
MLP, lr $= 10^{-3}$	7,5112	6,9146	0,10 %
AlexNet, lr $= 10^{-3}$	$\approx 6{,}9$	$\approx 6{,}91$	$\approx 0{,}08$ %
MLP, lr $= 10^{-4}$	$\approx 7{,}2$	$\approx 7{,}0$	$\approx 0{,}12$ %
AlexNet, lr $= 10^{-4}$	$\approx 6{,}91$	$\approx 6{,}91$	$\approx 0{,}10$ %

Observations de la première époque (20 000 images). Les valeurs précédées de $\approx$ sont lues sur les courbes TensorBoard ; la ligne MLP lr $= 10^{-3}$ provient du log console.

Cohérence avec la théorie.

— **AlexNet** démarre au niveau du hasard : loss $\approx 6{,}91 \approx \ln(1000)$ et accuracy $\approx 0{,}1$ %. À l'initialisation, les logits sont petits, la softmax est quasi uniforme et la loss vaut $\ln(1000)$.

— **MLP** : la loss d'entraînement de l'époque 1 est *supérieure* à $\ln(1000)$ (7,51 avec lr $= 10^{-3}$, $\approx 7{,}2$ avec lr $= 10^{-4}$). C'est une moyenne sur toute l'époque, qui inclut les premières itérations où les pas d'Adam sont grands par rapport à la taille de la première couche (154 587 entrées) : le réseau devient temporairement confiant et faux. La loss de validation revient ensuite vers 6,91 dès l'époque 1 ou 2.

— Avec lr $= 10^{-3}$, les deux modèles **restent collés à** $\ln(1000)$ **et à une accuracy de** $\approx 0{,}1$ % pendant tout l'entraînement : ils n'apprennent pas.

— 0,1 % correspond à 5 images correctes sur 5 000 : des écarts entre 0,08 % et 0,12 % relèvent du bruit statistique.

3.3 Comparaison MLP vs AlexNet sur 20 000 images

Configuration : `subset_size=20000` (5 000 images de validation), batch 32, Adam, 10 époques maximum, patience 5, GPU NVIDIA L4. Deux séries de runs :

— **série A** : lr $= 10^{-3}$ (runs `mlp_sub20000_lr0.001` et `alexnet_sub20000_lr0.001`) ;

— **série B** : lr $= 10^{-4}$ avec weight_decay $= 10^{-5}$, valeurs de l'énoncé (runs `mlp_sub20000_lr0.0001` et `alexnet_sub20000_lr0.0001`).

Remarques sur les captures : les accuracies sont affichées **en pourcentage** dans TensorBoard (0,6 sur l'axe signifie 0,6 %) ; le hasard vaut 0,1 %. Les courbes `alexnet` et `mlp` (sans suffixe, 2 époques) sont des tests rapides de validation du code sur 1 000 images (batch 16) et ne font pas partie de la comparaison.

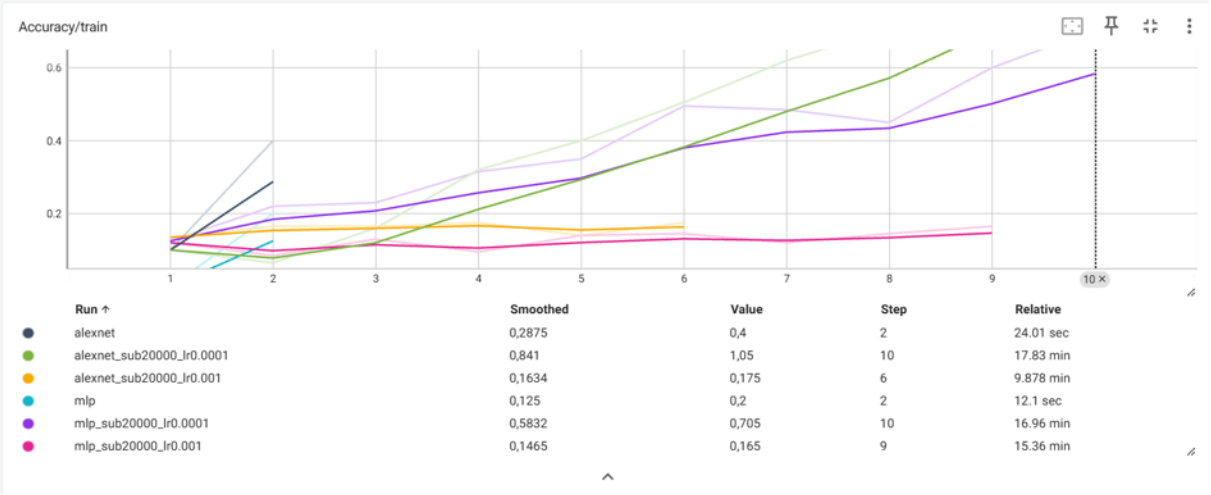
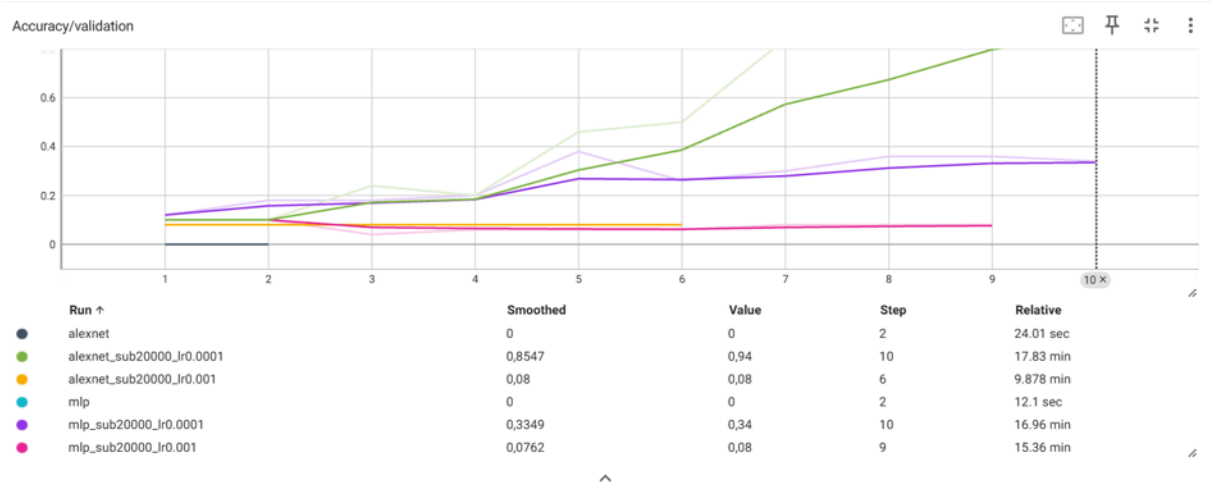
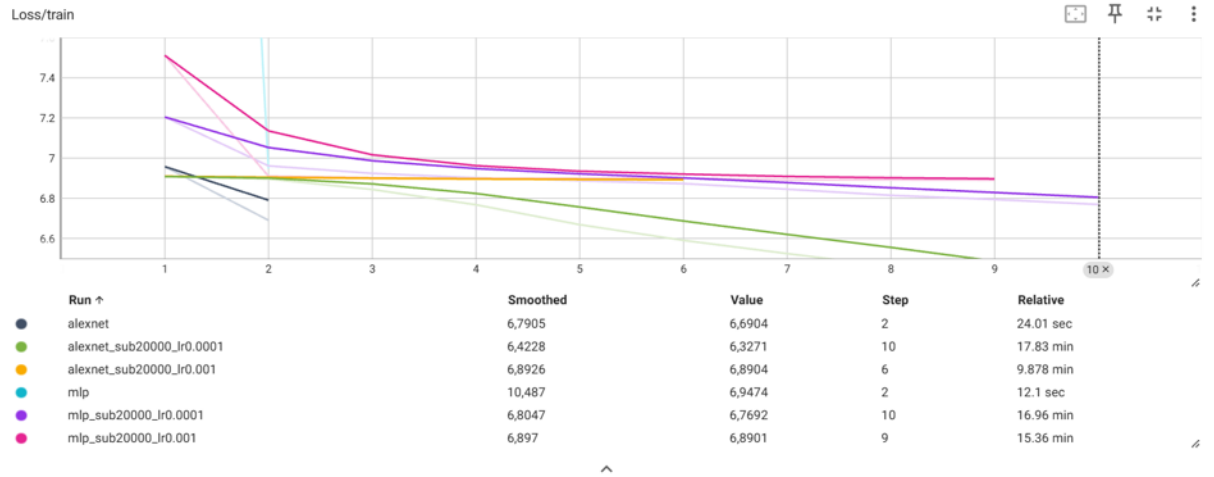


FIGURE 2 – Accuracy/train (séries A et B).



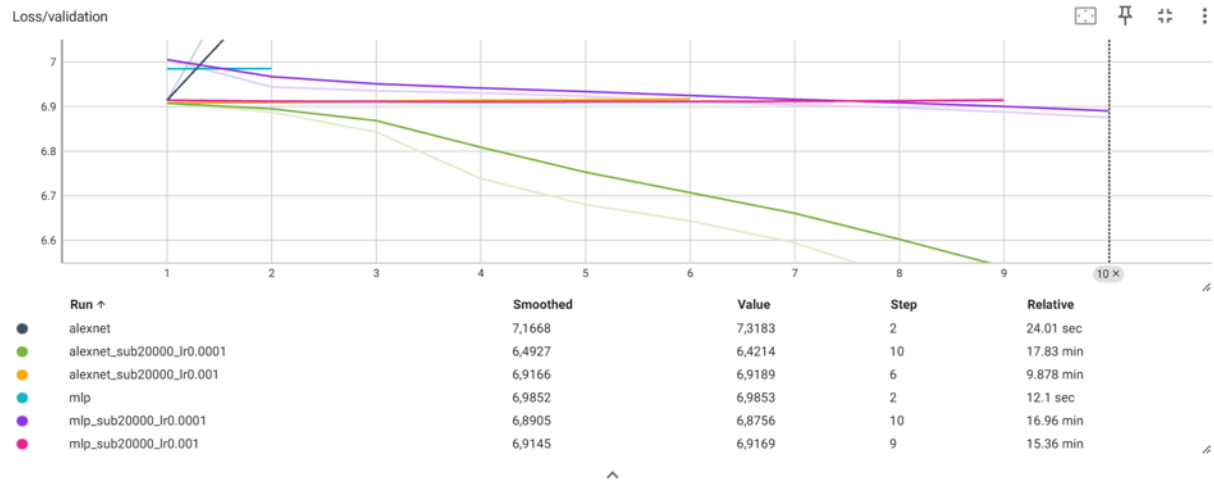
Run	Smoothed	Value	Step	Relative
alexnet	0	0	2	24.01 sec
alexnet_sub20000_lr0.0001	0,8547	0,94	10	17.83 min
alexnet_sub20000_lr0.001	0,08	0,08	6	9.878 min
mlp	0	0	2	12.1 sec
mlp_sub20000_lr0.0001	0,3349	0,34	10	16.96 min
mlp_sub20000_lr0.001	0,0762	0,08	9	15.36 min

FIGURE 3 – Accuracy/validation (séries A et B).



Run	Smoothed	Value	Step	Relative
alexnet	6,7905	6,6904	2	24.01 sec
alexnet_sub20000_lr0.0001	6,4228	6,3271	10	17.83 min
alexnet_sub20000_lr0.001	6,8926	6,8904	6	9.878 min
mlp	10,487	6,9474	2	12.1 sec
mlp_sub20000_lr0.0001	6,8047	6,7692	10	16.96 min
mlp_sub20000_lr0.001	6,897	6,8901	9	15.36 min

FIGURE 4 – Loss/train (séries A et B).



Run	Smoothed	Value	Step	Relative
alexnet	7,1668	7,3183	2	24.01 sec
alexnet_sub20000_lr0.0001	6,4927	6,4214	10	17.83 min
alexnet_sub20000_lr0.001	6,9166	6,9189	6	9.878 min
mlp	6,9852	6,9853	2	12.1 sec
mlp_sub20000_lr0.0001	6,8905	6,8756	10	16.96 min
mlp_sub20000_lr0.001	6,9145	6,9169	9	15.36 min

FIGURE 5 – Loss/validation (séries A et B).

Série A seule ($\mathrm{lr} = 10^{-3}$) et log console du MLP :

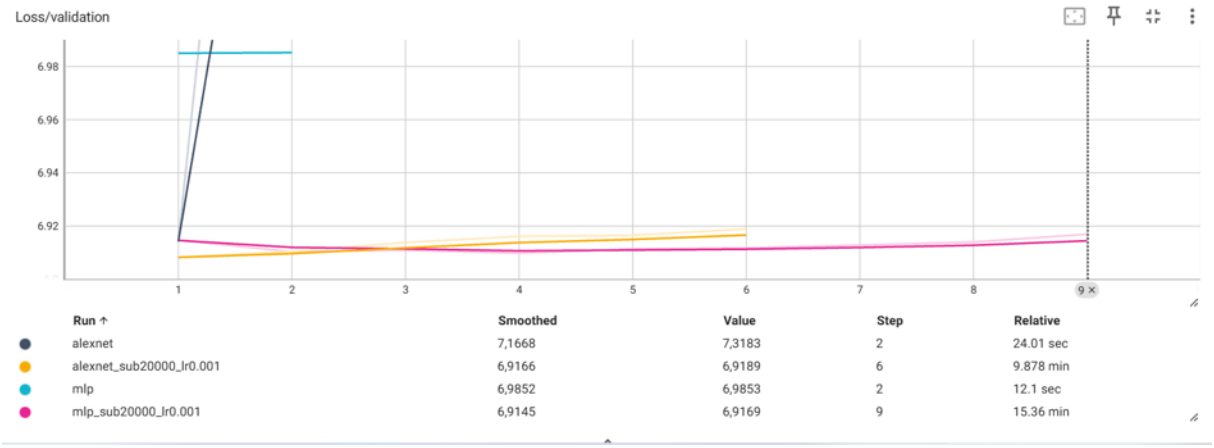


FIGURE 6 – Série A : Loss/validation.

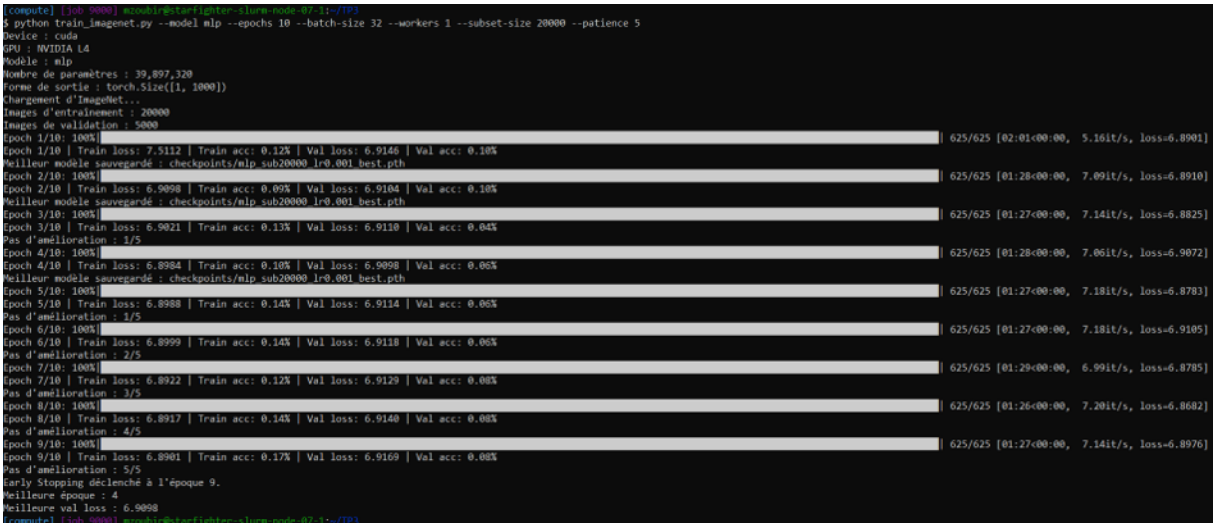


FIGURE 7 – Log console de l'entraînement du MLP ($\text{lr} = 10^{-3}$, Early Stopping à l'époque 9).

Run	Époques	Train loss	Val loss	Train acc	Val acc
MLP, $\text{lr} = 10^{-3}$	9 (Early Stopping ; meilleure époque 4, val loss 6,9098)	6,8901	6,9169	0,17 %	0,08 %
AlexNet, $\text{lr} = 10^{-3}$	6 (Early Stopping, d'après la courbe)	6,8904	6,9189	0,175 %	0,08 %
MLP, $\text{lr} = 10^{-4}$, wd 10^{-5}	10	6,7692	6,8756	0,705 %	0,34 %
AlexNet, $\text{lr} = 10^{-4}$, wd 10^{-5}	10	**6,3271**	**6,4214**	**1,05 %**	**0,94 %**

Valeurs finales.

Analyse.

1. $\text{lr} = 10^{-3}$ **: aucun des deux modèles n'apprend.** Les losses restent autour de 6,89–6,92 ($\approx \ln 1000$) et les accuracies à $\approx 0{,}1\,\%$, c'est-à-dire au niveau du hasard. La loss de validation remonte légèrement alors que celle d'entraînement baisse un peu, donc l'Early Stopping se déclenche (époque 9 pour le MLP, meilleure époque 4). Il ne s'agit pas d'un vrai

sur-apprentissage : le réseau est coincé sur un plateau où il prédit à peu près la distribution uniforme. Hypothèse : le learning rate est trop grand pour des réseaux sans BatchNorm (pas d'Adam trop larges qui détruisent le signal des premières itérations).

2. $\mathrm{lr} = 10^{-4}$ **: l'apprentissage démarre pour les deux.** Les courbes de loss, d'entraînement comme de validation, descendent de façon continue.
3. **AlexNet apprend nettement mieux que le MLP** : val loss 6,42 contre 6,88, val acc 0,94 % contre 0,34 % (soit environ 9× le hasard contre 3×). Avec 5 000 images de validation, l'incertitude de mesure vaut environ 0,14 point pour une accuracy de 1 % : l'écart de 0,6 point est significatif. La convolution apporte des biais inductifs adaptés aux images (localité, partage de poids, invariance par translation), alors que le MLP doit apprendre chaque motif séparément pour chaque pixel.
4. **Qui sur-apprend le plus vite ?** Le **MLP**, même si, en 10 époques, aucun des deux n'a encore une vraie remontée de la loss de validation avec $\mathrm{lr} = 10^{-4}$. Le signal est l'écart entre apprentissage et généralisation : le MLP atteint 0,705 % en train mais seulement 0,34 % en validation (il généralise moitié moins qu'il n'apprend), alors qu'AlexNet atteint 1,05 % en train et 0,94 % en validation (environ 90 % du niveau d'entraînement). L'explication est structurelle : 99 % des paramètres du MLP sont dans la première couche dense (39,6 M poids), où chaque pixel a ses propres poids, ce qui permet de mémoriser les 20 000 images sans apprendre de caractéristiques généralisables, et le MLP n'a pas de Dropout. AlexNet est régularisé par le partage de poids et par le Dropout de ses couches denses.
5. **Limite.** Avec ≈ 20 images par classe et 10 époques, aucun modèle n'a encore convergé (la loss d'AlexNet décroît toujours à l'époque 10 : l'Early Stopping ne s'est pas déclenché). Un entraînement plus long avec $\mathrm{lr} = 10^{-4}$ ferait apparaître le sur-apprentissage classique (train loss qui baisse, val loss qui remonte), d'abord pour le MLP.

Validation préalable du code. Un test sur 1 000 images (2 époques, batch 16) confirme que la boucle fonctionne (loss d'entraînement 6,96 puis 6,69).

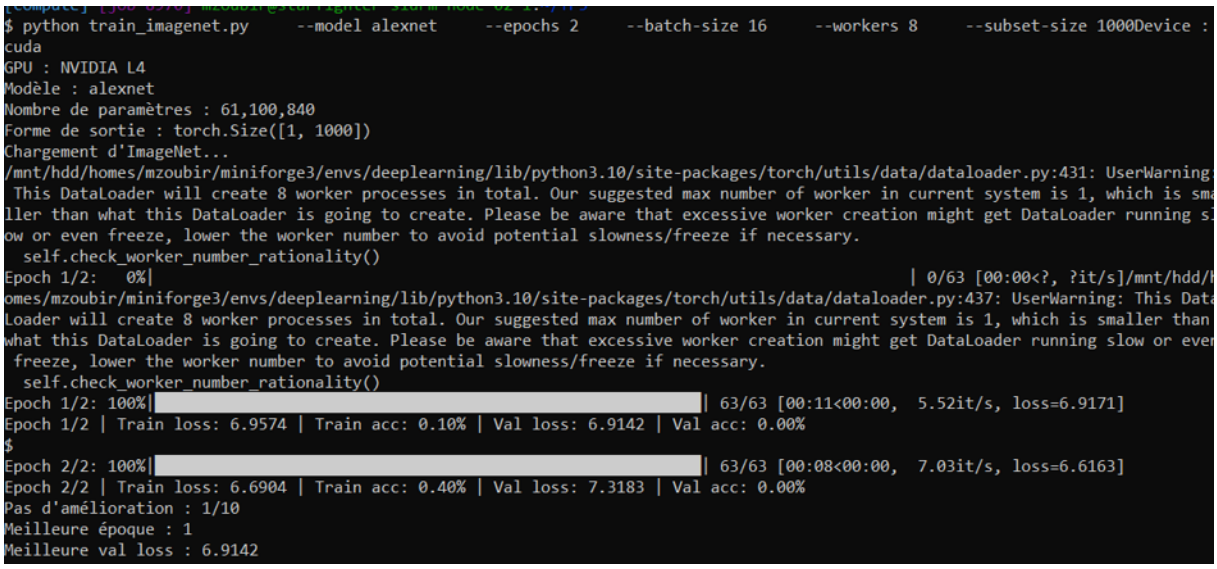


FIGURE 8 – Test rapide d'AlexNet sur 1 000 images.

3.4 Étape 2 : impact de la profondeur

Variante : trois convolutions supplémentaires `Conv2d(256, 256, kernel_size=3, padding=1)` suivies de ReLU, juste après Conv5 (la taille spatiale 13×13 est conservée). Chaque convolution ajoutée coûte $256\,(256 \times 9 + 1) = 590\,080$ paramètres, soit $+1\,770\,240$ au total.

```python
class AlexNetDeep(AlexNet):
    def __init__(self, num_classes=1000):
        super().__init__(num_classes)
        extra = []
        for _ in range(3):
            extra += [nn.Conv2d(256, 256, kernel_size=3, padding=1),
                      nn.ReLU(inplace=True)]
        # apres Conv5+ReLU (indice 11), avant MaxPool3 (indice 12)
        layers = list(self.features.children())
        self.features = nn.Sequential(*layers[:12], *extra, *layers[12:])
```

3.4.1 Sur-apprentissage ou sous-apprentissage ?

Si, comme attendu, la loss d'entraînement **reste bloquée près de** $\ln(1000) \approx 6{,}9$ ou décroît très lentement au début, il s'agit d'un **sous-apprentissage** (*underfitting*) et non d'un sur-apprentissage : le réseau n'arrive pas à ajuster même les données d'entraînement (le sur-apprentissage se reconnaît à une loss d'entraînement basse avec une loss de validation élevée).

Explication par la propagation du gradient. Le réseau est un empilement de couches ReLU sans normalisation (pas de BatchNorm) et sans connexions résiduelles. Le gradient est un produit de jacobiennes à travers toutes les couches :

$$\frac{\partial \mathcal{L}}{\partial W_1} = \frac{\partial \mathcal{L}}{\partial a_L} \prod_{l=2}^{L} \frac{\partial a_l}{\partial a_{l-1}} \cdot \frac{\partial a_1}{\partial W_1}.$$

Avec trois couches de plus, ce produit tend à se contracter (ou à exploser) de façon exponentielle avec la profondeur (*vanishing / exploding gradients*), d'autant plus que l'initialisation par défaut de PyTorch n'est pas calibrée pour ReLU (une initialisation de He serait mieux adaptée). Les premières couches reçoivent un gradient très faible : c'est un problème d'*optimisation*, pas de capacité du modèle. Pistes : initialisation de He, BatchNorm, connexions résiduelles (ResNet), learning rate adapté, warm-up.

4 Exercice 4 – Entraînement long via `sbatch`

Script `run_imagenet.sh`

```bash
#!/bin/bash
#SBATCH --partition=gpu
#SBATCH --gres=gpu:1
#SBATCH --cpus-per-task=8
#SBATCH --mem=24G
#SBATCH --time=04:00:00
#SBATCH -J alexnet-full
#SBATCH -o logs/%x-%j.out
#SBATCH -e logs/%x-%j.err

set -euo pipefail
mkdir -p logs

source ~/miniforge3/etc/profile.d/conda.sh
conda activate deeplearning

echo "Debut du job $SLURM_JOB_ID sur le noeud $SLURM_NODELIST"
nvidia-smi

python -u train_imagenet.py
```

Dans `train_imagenet.py` :

```
if __name__ == "__main__":
    train_model(model_name="alexnet", subset_size=None, num_epochs=50)
```

```
[compute] [job 9120] mzoubir@starfighter-slurm-node-01-1:~/TP3 (main *)
$ sbatch run_imagenet.sh
Submitted batch job 9122
[compute] [job 9120] mzoubir@starfighter-slurm-node-01-1:~/TP3 (main *)
$ squeue -u $USER
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
              9122       gpu alexnet-  mzoubir PD       0:00      1 (QOSMaxJobsPerUserLimit)
              9121       gpu alexnet-  mzoubir PD       0:00      1 (PartitionTimeLimit)
              9120       gpu      tp2  mzoubir  R       4:34      1 starfighter-slurm-node-01-1
[compute] [job 9120] mzoubir@starfighter-slurm-node-01-1:~/TP3 (main *)
$ scancel 9121
[compute] [job 9120] mzoubir@starfighter-slurm-node-01-1:~/TP3 (main *)
$ exit           # ferme la session 9120 ; vous revenez sur le nœud de connexion
exit
[controller] mzoubir@arcadia-slurm-controller:~
$ squeue -u $USER
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
              9122       gpu alexnet-  mzoubir  R       0:09      1 starfighter-slurm-node-01-1
[controller] mzoubir@arcadia-slurm-controller:~
$
```

FIGURE 9 – `sbatch run_imagenet.sh`, puis `squeue -u $USER` (état `R`).

4.1 Profilage temporel et matériel

```
[controller] mzoubir@arcadia-slurm-controller:~/TP3 (main *)
$ cd ~/TP3
[controller] mzoubir@arcadia-slurm-controller:~/TP3 (main *)
$ tr '\r' '\n' < logs/alexnet-full-9122.err | tail -n 2
Epoch 1/50:  36%|███       | 14398/40037 [08:45<15:01, 28.44it/s, loss=6.9086]
[controller] mzoubir@arcadia-slurm-controller:~/TP3 (main *)
```

FIGURE 10 – Profilage temporel

```
[controller] mzoubir@arcadia-slurm-controller:~/TP3 (main *)
$ srun --jobid=9122 --overlap nvidia-smi --query-gpu=utilization.gpu,memory.used --format=csv
utilization.gpu [%], memory.used [MiB]
98 %, 1582 MiB
[controller] mzoubir@arcadia-slurm-controller:~/TP3 (main *)
$
```

FIGURE 11 – GPU USAGE

4.1.1 Pourquoi le sur-apprentissage est-il beaucoup plus faible que sur 20 000 images ?

— **Quantité de données** : 1,28 M images contre 20 000 (environ 64× plus, soit ≈ 1 280 images par classe contre ≈ 20). Le rapport « nombre d'exemples / nombre de paramètres » est bien meilleur : le réseau ne peut plus mémoriser le jeu d'entraînement aussi facilement.

— **Augmentation de données** : `RandomResizedCrop` et le flip produisent à chaque époque une vue différente de chaque image ; sur l'ensemble complet, le réseau ne voit quasiment jamais deux fois exactement la même entrée.

— **Dropout** ($p = 0{,}5$) dans les couches denses, qui concentrent 96 % des paramètres, et **weight decay** (10^{-5}).

— Après quelques époques seulement, le réseau est encore loin de la phase de mémorisation : les losses d'entraînement et de validation décroissent ensemble.

4.1.2 Pourquoi la loss de validation est-elle parfois inférieure à la loss d'entraînement ?

Deux raisons liées à notre code :

1. **Le Dropout est actif en entraînement (`model.train()`) mais désactivé en validation (`model.eval()`).** En entraînement, la moitié des neurones des couches denses est éteinte aléatoirement : le réseau perturbé obtient une loss plus élevée que le réseau complet utilisé en validation.
2. **L'augmentation de données n'est appliquée qu'à l'entraînement.** `RandomResizedCrop` (qui peut ne garder qu'une petite partie de l'image, parfois sans l'objet) et le flip rendent les exemples d'entraînement plus difficiles que les images de validation, obtenues par `Resize(256)` + `CenterCrop(227)`, plus « propres » et centrées.

À cela s'ajoute un effet de mesure : la loss d'entraînement est une moyenne cumulée sur toute l'époque, calculée avec des poids qui évoluent (donc ceux, moins bons, du début d'époque), alors que la loss de validation est calculée en fin d'époque avec les poids les plus récents.